U11 · Bases de datos orientadas a objetos

En la unidad 10 viste que guardar un objeto en una base de datos relacional exige "desmontarlo" en filas y columnas (y volver a montarlo al leerlo). Esta unidad plantea la pregunta contraria: **¿y si la base de datos guardara los objetos tal cual son?**

1. Del desfase objeto-relacional a las bases de datos orientadas a objetos

Recuerda el **desfase objeto-relacional**: tus clases tienen herencia, referencias a otros objetos y colecciones; las tablas tienen claves ajenas y filas planas. Un ORM (JPA, Hibernate) traduce entre ambos mundos, pero esa traducción es trabajo extra, cuesta rendimiento y se complica cuanto más rico es tu modelo de clases.

Una **base de datos orientada a objetos** (BDOO) elimina el problema de raíz: el modelo de la base de datos **es** el modelo de objetos. Sus características principales:

- **No hay diferencia de modelo**: los objetos se guardan con su estructura real, sin mapeos ni tablas intermedias.
- **Identidad de objeto**: cada objeto tiene una identidad propia, independiente de sus valores (como una referencia en memoria, pero persistente).
- **Soporta herencia, polimorfismo y objetos complejos** (objetos dentro de objetos, colecciones de objetos) de forma natural.
- **Esquema automático**: la base de datos se adapta a tus clases; no hay `CREATE TABLE`.
- **Consultas orientadas a objetos**: en vez de SQL, se consulta con objetos y con el propio lenguaje (el estándar histórico, ODMG, definía un lenguaje parecido a SQL llamado **OQL**).

> **¿Por qué no se usan más?**
>
> Las BDOO nunca desplazaron a las relacionales: se quedaron en nichos concretos (software embebido, ingeniería, datos muy complejos). Las bases de datos relacionales ya eran un estándar enorme, con herramientas, SQL y administradores por todas partes, y los ORM terminaron de resolver bien el problema. Aun así, estudiarlas es útil: te enseñan **cómo sería** la persistencia ideal para un programador de objetos, y varias ideas (persistencia transparente, consultas con el lenguaje) han acabado en otras tecnologías.

2. Operaciones básicas con una BDOO: db4o

Como caso de estudio usaremos **db4o**, un motor de base de datos orientado a objetos **embebido** que se distribuye como un único fichero `.jar`: la base de datos es un fichero que tu propio programa abre, sin servidor que instalar.

> **db4o está abandonado**
>
> db4o dejó de mantenerse hace años: no recibe actualizaciones ni correcciones de seguridad, y conseguir el `.jar` puede ser complicado. No lo elegirías para un proyecto nuevo. Lo estudiamos porque su API es **muy limpia** y muestra con claridad todas las ideas de una BDOO; los conceptos (almacenar, consultar por ejemplo, actualizar con profundidad, borrado en cascada) son los mismos en cualquier otra. Si no consigues el `.jar`, puedes seguir esta sección igualmente para entender las ideas, y hacer la práctica con MongoDB (apartados 5 y 6).

Usaremos una clase sencilla, sin herencia ni referencias a otros objetos:

```java
public class Piloto {
    private String nombre;
    private int puntos;

    public Piloto(String nombre, int puntos) {
        this.nombre = nombre;
        this.puntos = puntos;
    }
    public String getNombre() { return nombre; }
    public int getPuntos() { return puntos; }
    public void sumaPuntos(int p) { puntos += p; }
    @Override public String toString() { return nombre + "/" + puntos; }
}
```

Fíjate en lo que **no** hay: ni anotaciones, ni constructor especial, ni interface que implementar. La clase es un objeto Java corriente.

Abrir, almacenar y cerrar

```java
ObjectContainer db = Db4oEmbedded.openFile(Db4oEmbedded.newConfiguration(),
"F1.db4");
try {
    db.store(new Piloto("Michael Schumacher", 100));
    db.store(new Piloto("Rubens Barrichello", 99));
} finally {
    db.close();            // imprescindible: cierra el fichero y confirma los
cambios pendientes
}
```

`openFile` abre el fichero o lo **crea si no existe**. El `ObjectContainer` es tu base de datos, y `store(objeto)` guarda el objeto entero, con todos sus campos.

Recuperar objetos: tres formas de consultar

1. Consulta por ejemplo (QBE, *Query By Example*): das un objeto "plantilla" y la base de datos devuelve los que se le parecen. Los campos con valor por defecto (`null`, `0`, `false`) se **ignoran**.

```
ObjectSet<Piloto> todos = db.queryByExample(Piloto.class);                  //
todos los pilotos
ObjectSet<Piloto> uno = db.queryByExample(new Piloto("Michael Schumacher", 0));   //
solo por nombre
```

> ⚠ **El 0 no cuenta**
>
> En el ejemplo anterior, los `0` puntos significan "no me importa cuántos puntos tenga", **no** "que tenga 0 puntos". Por eso QBE no sirve para buscar un campo cuyo valor sea precisamente el valor por defecto.

2. Consultas nativas (*Native Queries*): escribes la condición **en Java**, con un objeto `Predicate` cuyo método `match` devuelve `true` para los objetos que quieres. Es la forma recomendada, porque el compilador comprueba los tipos y los nombres de los campos:

```
List<Piloto> buenos = db.query(new Predicate<Piloto>() {
    @Override
    public boolean match(Piloto p) {
        return p.getPuntos() > 99 || p.getNombre().equals("Rubens Barrichello");
    }
});
```

3. SODA: una API de bajo nivel en la que construyes la consulta como un **grafo de nodos**, usando nombres de campo como `String`. Es más incómoda y el compilador no revisa los nombres, pero es la única opción cuando la consulta se construye **dinámicamente** (no la conoces al compilar):

```
Query q = db.query();
q.constrain(Piloto.class);
q.descend("puntos").constrain(99).greater();       // puntos > 99
q.descend("nombre").orderAscending();
ObjectSet<Piloto> resultado = q.execute();
```

Actualizar y borrar

Para **actualizar**, primero **recupera** el objeto, modifícalo y vuelve a hacer `store`. Si lo creas nuevo con los mismos datos, la base de datos no sabe que es "el mismo" y lo guarda como un

segundo objeto distinto.

```
Piloto p = db.queryByExample(new Piloto("Michael Schumacher", 0)).next();
p.sumaPuntos(11);
db.store(p);               // actualiza el existente, porque venía de la base de
datos
```

Para **borrar**, `db.delete(objeto)` (también sobre un objeto recuperado antes):

```
db.delete(p);
```

3. Operaciones avanzadas

Objetos compuestos y la profundidad de las operaciones

Añadimos una clase `Coche` que **contiene** un `Piloto`. Guardar un coche guarda también su piloto, sin que tengas que hacer nada:

```
Coche ferrari = new Coche("Ferrari", new Piloto("Fernando Alonso", 0));
db.store(ferrari);
```

Pero hay un detalle importante. Si en otra sesión recuperas el coche, le sumas puntos a **su piloto** y haces `store(coche)`, la base de datos **no guarda el cambio del piloto**: por defecto, al actualizar un objeto solo se actualizan sus campos simples (números, `String`), no los objetos que contiene. Se debe a que actualizar en profundidad cuesta, y la base de datos no sabe si lo necesitas. La solución es pedirlo explícitamente en la configuración:

```
EmbeddedConfiguration cfg = Db4oEmbedded.newConfiguration();
cfg.common().objectClass(Coche.class).cascadeOnUpdate(true);   // actualiza también
los objetos contenidos
ObjectContainer db = Db4oEmbedded.openFile(cfg, "F1.db4");
```

Lo mismo ocurre al **borrar**: `db.delete(coche)` elimina el coche pero **deja el piloto huérfano** en la base de datos. Con `cascadeOnDelete(true)` se borra también el piloto contenido.

> **La misma idea en todos sitios**
>
> Decidir hasta dónde "llega" una operación sobre un objeto que contiene a otros es un problema que aparece en cualquier tecnología de persistencia. En JPA se resuelve con las opciones `cascade = CascadeType.PERSIST / REMOVE...` de las anotaciones de relación. Cambia la sintaxis, la pregunta es idéntica.

Colecciones y herencia

Un objeto puede contener **arrays y colecciones** de otros objetos (una `List<Piloto>` dentro de un `Equipo`), y las consultas pueden filtrar por su contenido. Y como la base de datos guarda objetos reales, la **herencia y el polimorfismo funcionan**: una consulta sobre la superclase `Vehiculo` devuelve también los `Coche` y `Moto` almacenados, sin que tengas que pensar en cómo mapear una jerarquía a tablas (en el modelo relacional, esa decisión tiene tres estrategias distintas, cada una con sus pegas).

Transacciones

Igual que con JDBC, un `ObjectContainer` trabaja dentro de una transacción: los cambios no son definitivos hasta `commit()`, y `rollback()` los deshace. Al cerrar la base de datos se hace `commit` automáticamente.

```
db.store(new Piloto("Nuevo", 5));
db.rollback();              // el piloto nuevo no llega a guardarse
```

Persistencia transparente

Cuando recuperas un objeto con muchos objetos anidados, traer **todo el grafo** a memoria sería carísimo. Por eso las BDOO cargan los objetos hasta cierta **profundidad de activación** (por defecto unos pocos niveles) y los más profundos se cargan solo cuando los necesitas. La **persistencia transparente** automatiza ese proceso: las clases implementan una interface sencilla y la base de datos activa y guarda lo necesario sin que tengas que pensar en ello.

4. ¿Y si quiero una BDOO que sí se mantenga?

Existen productos orientados a objetos que siguen vivos, como **ObjectDB**, que se maneja con el API estándar **JPA** que viste en la unidad 10: las mismas anotaciones (`@Entity`, `@Id`) y las mismas operaciones (`persist`, `find`, consultas JPQL), pero la base de datos guarda los objetos directamente en vez de pasar por tablas. Es una opción muy cómoda si ya conoces JPA, aunque su licencia y su edición gratuita tienen limitaciones que conviene consultar antes de usarla en un proyecto.

5. Introducción a las bases de datos NoSQL

NoSQL ("*Not Only SQL*") agrupa a todas las bases de datos que **no** son relacionales. Nacieron para necesidades que las relacionales cubrían mal: cantidades enormes de datos repartidos en muchos servidores, estructuras de datos que cambian constantemente, o lecturas y escrituras muy rápidas. Hay varias familias:

Tipo	Cómo guarda los datos	Ejemplos	Uso típico
Documentos	Documentos tipo JSON, con estructura flexible	MongoDB, CouchDB	Catálogos de productos, contenidos, perfiles de usuario
Clave-valor	Un valor por clave, como un gran `HashMap`	Redis	Cachés, sesiones, contadores
Columnas anchas	Tablas con columnas flexibles, repartidas en muchos nodos	Cassandra	Enormes volúmenes de datos (registros, series temporales)
Grafos	Nodos y relaciones entre ellos	Neo4j	Redes sociales, recomendaciones, rutas

La diferencia de fondo: las relacionales garantizan **ACID** (atomicidad, consistencia, aislamiento, durabilidad) y un esquema estricto; muchas NoSQL renuncian a parte de esas garantías (por ejemplo, aceptando que los datos tarden un instante en ser consistentes en todos los servidores) a cambio de escalar más fácilmente y de más flexibilidad. **No es que una sea mejor**: depende del problema.

6. MongoDB

MongoDB es la base de datos NoSQL de documentos más extendida. Sus conceptos se corresponden de forma aproximada con los relacionales:

Relacional	MongoDB
Base de datos	Base de datos
Tabla	**Colección**
Fila	**Documento**
Columna	**Campo**

Un **documento** es un objeto en formato similar a JSON (internamente BSON), con campos de cualquier tipo, **incluidos otros documentos y listas**. Dos documentos de la misma colección **no tienen por qué tener los mismos campos**:

```
{
  "_id": "ObjectId(...)",
  "nombre": "Rafael",
  "genero": "pop",
  "likes": 100,
  "discos": [
    { "titulo": "Mi gran noche", "año": 1966 },
    { "titulo": "Cuando tú no estás", "año": 1969 }
  ]
}
```

Todo documento tiene un campo **`_id`** único en su colección; si no lo indicas, MongoDB genera un `ObjectId` (un identificador de 12 bytes casi imposible de repetir). Para acceder a partes anidadas se usa la **notación de punto**: `discos.0.titulo`.

Relaciones: embeber o referenciar

Sin `JOIN`s al estilo relacional, la pregunta de diseño es cómo relacionar los datos:

- **Embeber**: guardar el documento relacionado **dentro** del otro (los `discos` de un cantante). Lectura muy rápida (todo en una consulta) y las escrituras sobre un documento son atómicas. Conviene en relaciones de “contiene” o uno-a-pocos.

- **Referenciar**: guardar solo el `_id` del otro documento, como una clave ajena. Evita duplicar datos y sirve para relaciones muchos-a-muchos, pero exige una segunda consulta para obtener lo referenciado.

Operaciones desde el shell de MongoDB

```
db.cantantes.insertOne({ nombre: "Rafael", genero: "pop", likes: 100 })
db.cantantes.insertMany([{ nombre: "AC/DC", genero: "heavy", likes: 200 }, { nombre:
"Mecano", genero: "pop", likes: 150 }])

db.cantantes.find()                                  // todos
db.cantantes.find({ genero: "pop" })                 // filtro por igualdad
db.cantantes.find({ likes: { $gt: 120 } })           // operador: mayor que
db.cantantes.find({ genero: "pop" }).sort({ likes: -1 }).limit(5)

db.cantantes.updateOne({ nombre: "Rafael" }, { $set: { likes: 150 } })
db.cantantes.updateMany({ genero: "pop" }, { $inc: { likes: 10 } })

db.cantantes.deleteOne({ nombre: "AC/DC" })
```

Los **operadores** empiezan por `$`: de comparación (`$gt`, `$lt`, `$gte`, `$ne`, `$in`), lógicos (`$and`, `$or`), de actualización (`$set`, `$inc`, `$push`)… Para consultas más complejas (agrupar, sumar, contar) existe el *aggregation framework*, y para acelerar búsquedas se crean **índices** (`db.cantantes.createIndex({ nombre: 1 })`).

7. Usar MongoDB desde Java

Necesitas el driver oficial, `org.mongodb:mongodb-driver-sync` (añádelo como dependencia Maven o desde *Project Structure → Libraries → From Maven*, igual que hiciste con el driver JDBC):

```java
import com.mongodb.client.*;
import com.mongodb.client.model.*;
import org.bson.Document;

try (MongoClient cliente = MongoClients.create("mongodb://localhost:27017")) {
    MongoDatabase bd = cliente.getDatabase("prueba");               // si no existe,
se crea al escribir en ella
    MongoCollection<Document> cantantes = bd.getCollection("cantantes");   // ídem

    // Insertar
    cantantes.insertOne(new Document("nombre", "Rafael").append("genero",
"pop").append("likes", 100));

    // Consultar: filtros con la clase Filters
    for (Document d : cantantes.find(Filters.gt("likes",
50)).sort(Sorts.descending("likes"))) {
        System.out.println(d.getString("nombre") + " - " + d.getInteger("likes"));
    }

    // Actualizar y borrar
    cantantes.updateOne(Filters.eq("nombre", "Rafael"), Updates.set("likes", 150));
    cantantes.deleteOne(Filters.eq("nombre", "Rafael"));
}
```

- `MongoClient` se cierra con `try-with-resources`, como una conexión JDBC.
- Un `Document` es, básicamente, un `Map<String, Object>` con métodos cómodos (`getString`, `getInteger`…).
- Las clases `Filters` (`eq`, `gt`, `and`, `or`…), `Updates` (`set`, `inc`…) y `Sorts` construyen los filtros y modificaciones sin escribir JSON a mano.

> **Es la misma idea que JDBC, simplificada**
>
> Conectar → obtener la colección (la "tabla") → ejecutar operaciones → cerrar. La diferencia es que aquí no hay sentencias SQL ni `ResultSet`: los resultados son documentos, y las condiciones se construyen con objetos Java.

8. ¿Qué elijo?

Si tu problema es…	Piensa en…
Datos muy estructurados, relaciones claras, necesidad de transacciones y consultas complejas	Una base de datos **relacional** (unidad 10)
Datos con estructura cambiante y muchas lecturas de “documentos completos”	Una base de datos de **documentos** (MongoDB)
Un modelo de objetos muy rico que quieres persistir sin traducir	Una **BDOO**, o un ORM sobre una relacional
Cachés y datos temporales de acceso ultrarrápido	**Clave-valor**

La respuesta real casi siempre es “depende”, y muchas aplicaciones combinan varias a la vez (la relacional como fuente de verdad, una clave-valor como caché).

RA y CE que cubre esta unidad

RA	Criterios de evaluación cubiertos
RA8. Utiliza bases de datos orientadas a objetos, analizando sus características y aplicando técnicas para mantener la persistencia de la información.	a) características de las BDOO · b) su aplicación en el desarrollo con lenguajes orientados a objetos · c) instalación de sistemas gestores · d) métodos soportados por los gestores para la información almacenada · e) crear bases de datos y estructuras de almacenamiento de objetos · f) aplicaciones que almacenan objetos · g) programas para recuperar, actualizar y eliminar objetos · h) tipos de datos estructurados, compuestos y relacionados

🟢 Boletín inicial ⭐ Boletín intermedio 🔥 Extras